

From Pruning Weights to Preserving Evidence

OBD / OBS, sparsity learning, and structured ViT pruning

Seung Ho Go

September 6, 2026

Paper-review seminar — Pruning → XAI → Ongoing research

Which parts can we remove — and what should survive?

One question, three levels

A smaller network should preserve more than its size advantage.

- 1 **Prediction:** Does deleting this part increase the loss?
- 2 **Computation:** Does deleting it produce a genuinely smaller model?
- 3 **Evidence:** Does the model still rely on useful input cues?

Today: pruning and KD (about 20 min), then shortcuts, XAI and EviPrune (about 8 min).

Six papers, three changes in the pruning question

	Paper	What changes?
1989/92	OBD $\rightarrow$ OBS	From weight size to estimated loss damage
2015	Weights & Connections	Make magnitude pruning practical: train, prune, retrain
2017	Network Slimming	Learn which channels to remove
2018	L_0 Regularization	Learn stochastic gates with a sparsity objective
2023	NViT	Rank ViT structural groups globally and account for latency

Not a complete history: a path from **importance scores** to **learned structure**.

A zero weight is not yet a smaller matrix

Unstructured pruning

$$W = \begin{bmatrix} w_{11} & 0 & w_{13} \\ 0 & w_{22} & 0 \\ w_{31} & w_{32} & 0 \end{bmatrix}$$

- Remove individual connections.
- Matrix shape stays the same.
- Speedup depends on sparse execution.

Structured pruning

$$H_{N \times m} = \sigma(X_{N \times d} W_{1, d \times m} + b_1)$$

$$Y_{N \times d} = H_{N \times m} W_{2, m \times d} + b_2$$

N : tokens, d : input/output width, m : hidden width.

- Remove one hidden unit j as a group.
- Delete column j of W_1 , bias b_{1j} , and row j of W_2 .
- Hidden width: $m \rightarrow m - 1$.

Masking changes values. **Physical pruning** changes tensor dimensions.

Estimate the damage before deleting a weight

$$\Delta\mathcal{L} = \mathcal{L}(w + \delta w) - \mathcal{L}(w) \approx \underbrace{g^\top \delta w}_{\text{local slope}} + \frac{1}{2} \underbrace{\delta w^\top H \delta w}_{\text{local curvature}}$$

The symbols

- w : all trainable weights.
- $g = \nabla_w \mathcal{L}$: gradient.
- $H = \nabla_w^2 \mathcal{L}$: Hessian.

The pruning move

- Delete w_i : set $\delta w_i = -w_i$.
- Near a trained local optimum: $g \approx 0$.
- Curvature predicts how costly this move is.

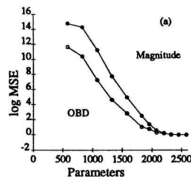
A small weight can still sit in a **sensitive direction**.

OBD and OBS: deletion with or without compensation

Optimal Brain Damage

$$s_i = \frac{1}{2} H_{ii} w_i^2$$

Diagonal curvature estimates the damage of an isolated deletion. Retrain after pruning.

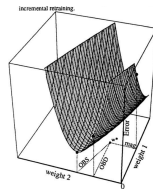


OBD vs. magnitude in the original experiment.

Optimal Brain Surgeon

$$s_i = \frac{w_i^2}{2[H^{-1}]_{ii}}$$

Inverse curvature finds a deletion that other weights can compensate for. More costly to compute.



The remaining weights move on the error surface.

Low score = low estimated damage. Both rely on a **local loss approximation**.

Han et al.: train, prune, retrain — and repeat

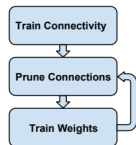


Figure 2: Three-Step Training Pipeline.

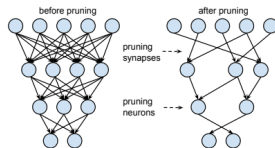


Figure 3: Synapses and neurons before and after pruning.

- ① Train a dense network.
- ② Remove small-magnitude connections.
- ③ Retrain retained connections.

Model	Parameters before / after	Reduction	Top-1 error before / after
AlexNet	61M / 6.7M	9×	42.78% / 42.77%
VGG-16	138M / 10.3M	13×	31.50% / 31.34%

The practical recipe is **selection plus recovery**, not thresholding alone.

Weight decay shrinks weights; pruning deletes them

$$J(w) = \mathcal{L}_{\text{data}}(w) + \frac{\lambda}{2} \|w\|_2^2$$

$$w^{t+1} = w^t - \eta(\nabla \mathcal{L}_{\text{data}}(w^t) + \lambda w^t) = \boxed{(1 - \eta\lambda)w^t - \eta \nabla \mathcal{L}_{\text{data}}(w^t)}$$

If the data gradient were zero

$$w^t = (1 - \eta\lambda)^t w^0$$

For $0 < 1 - \eta\lambda < 1$, a nonzero value shrinks but does not become exactly zero in finitely many idealized steps.

An explicit pruning decision

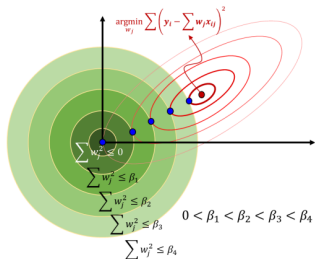
$$m_i = \mathbf{1}\{|w_i| \geq \tau\}, \quad w'_i = m_i w_i$$

A threshold adds a separate keep/delete decision. Regularization can influence the weights being ranked.

Shrinkage $\neq$ sparsity $\neq$ physical removal.

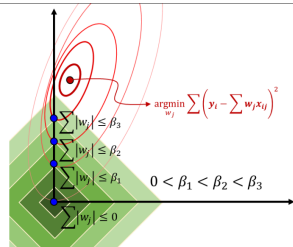
From class: why L1 is more naturally sparse than L2

L2: a smooth constraint boundary



$$\sum_j w_j^2 \leq B$$

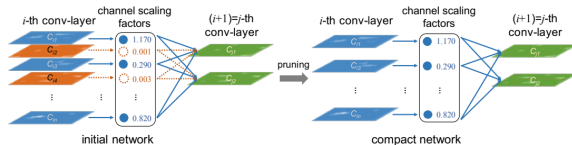
L1: corners lie on the axes



$$\sum_j |w_j| \leq B$$

An optimum on an L1 corner can set coordinates to **exactly zero**.

Network Slimming: apply L1 to a channel's scale



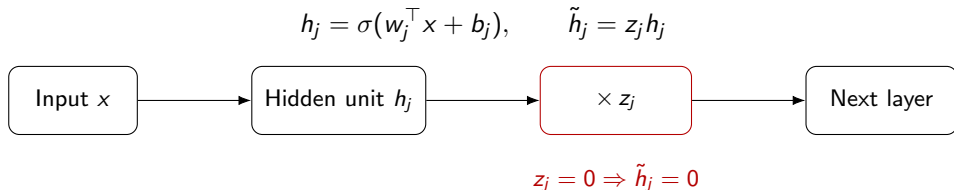
$$\hat{x}_c = \frac{x_c - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}, \quad y_c = \underbrace{\gamma_c}_{\text{channel scale}} \hat{x}_c + \beta_c$$

$$J = \mathcal{L}_{\text{task}} + \lambda \sum_c |\gamma_c|$$

- Use existing batch-normalization scales as channel-level selection variables.
- Threshold small $|\gamma_c|$, remove corresponding channels, then fine-tune.

Do not shrink every connection independently: **shrink a whole channel's scale.**

A gate can turn many connections into one decision



- **Placement defines the pruning unit:** weight, neuron, channel, or head.
- Train with gates; export a smaller network by removing permanently inactive groups.
- A BN scale is gate-like, but $\gamma_c = 0$ leaves $y_c = \beta_c$: handle the offset.

A zero gate is an algebraic decision; **export** turns it into smaller computation.

L0 regularization: pay for the connections that exist

$$\|\theta\|_0 = \sum_j \mathbf{1}\{\theta_j \neq 0\}, \quad J = \mathcal{L}(\theta) + \lambda \|\theta\|_0$$

Problem: the nonzero count does not give a useful ordinary gradient.

$$\theta_j = \tilde{\theta}_j z_j, \quad z_j \sim \text{Bernoulli}(\pi_j)$$

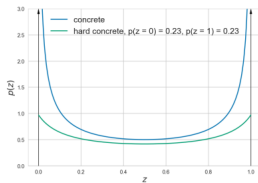
$$\min_{\tilde{\theta}, \pi} \mathbb{E}_z[\mathcal{L}(\tilde{\theta} \odot z)] + \lambda \sum_j \pi_j$$

Weights learn *what to compute*.

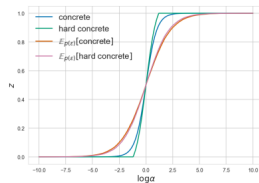
Gate probabilities learn *whether to compute it*.

Replace a hard count by an **expected active-gate count**, then make gates trainable.

Hard concrete: smooth inside, exactly zero at the edge



(a)



(b)

$$u \sim U(0, 1) \longrightarrow s \in (0, 1) \longrightarrow \bar{s} \in (\gamma, \zeta) \longrightarrow \boxed{z = \text{clip}_{[0,1]}(\bar{s})}$$

- Stretch beyond $[0, 1]$ ($\gamma < 0, \zeta > 1$), then clip: probability mass at 0 and 1.
- Learn the distribution parameter $\log \alpha$ through a reparameterized estimator.
- Training gates can also lie *between* 0 and 1; they are not purely binary.

The objective uses $\sum_j P(z_j > 0)$, **not** $\sum_j \mathbb{E}[z_j]$ for continuous gates.

How can one scalar loss learn many different gates?

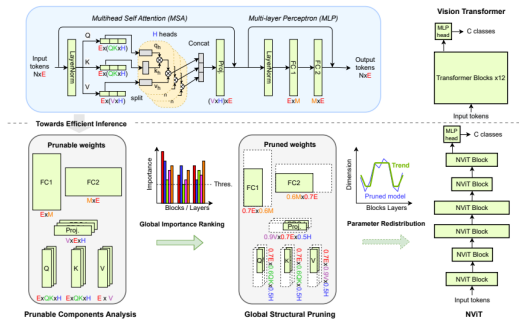
$$J = \mathbb{E}_z[\mathcal{L}(\tilde{\theta} \odot z)] + \lambda \sum_j P(z_j > 0)$$

$$\frac{\partial J}{\partial \log \alpha_j} = \underbrace{\frac{\partial \mathbb{E}[\mathcal{L}]}{\partial \log \alpha_j}}_{\text{task-specific pressure}} + \lambda \underbrace{\frac{\partial P(z_j > 0)}{\partial \log \alpha_j}}_{\text{pressure to turn off}}$$

- One scalar objective has a **different partial derivative for each gate**.
- A useful group can justify its cost; a redundant group may not.
- At test time, compute deterministic gates; physically remove zero-gated groups.

Learn a trade-off, not a hand-written rule saying “delete neuron 7.”

NViT: search across the structural dimensions of a ViT



Five prunable dimensions

- Embedding width E
- Number of heads H
- Query/key width Q/K
- Value width V
- MLP hidden width

Rank groups globally instead of imposing the same pruning ratio on every layer.

This is **structural pruning**, not input-dependent token dropping.

NViT: a group score and a hardware-aware correction

The implemented importance score for a group S

$$I_S = \left(\sum_{s \in S} \frac{\partial \mathcal{L}}{\partial w_s} w_s \right)^2 = \left(\frac{\partial \mathcal{L}}{\partial g_S} \right)_{g_S=1}^2$$

g_S is an explanatory shared gate multiplying the weights in S .

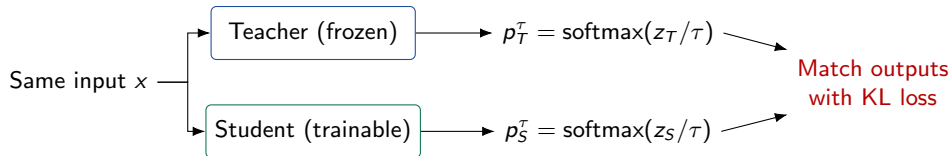
Latency-aware ranking

$$I_S^L = I_S - \underbrace{\eta [\text{Lat}(W) - \text{Lat}(W \setminus S)]}_{\text{estimated latency reduction}}$$

- Low importance and large latency savings make removal attractive.
- “Hessian-aware” motivates the derivation; the final score uses gradient-weight products.

NViT does **not** compute the OBS inverse-Hessian deletion score.

Knowledge distillation: learning from a teacher's outputs



Target	Cat	Dog	Car
Hard label	1.0	0.0	0.0
Teacher example	0.8	0.2	0.0

Soft targets can convey relations between classes. Higher τ makes the distribution softer.

$$\mathcal{L}_{\text{student}} = \underbrace{a \text{CE}(y, p_S)}_{\text{learn from labels}} + \underbrace{b \tau^2 \text{KL}(p_T^\tau \| p_S^\tau)}_{\text{learn from the teacher}}$$

The student learns class probabilities as well as the correct label.

NViT: output matching during pruning and recovery

Prune and fine-tune with teacher supervision

$$\mathcal{L} = \mathcal{L}_{\text{CNN}} + \alpha \mathcal{L}_{\text{full}}$$

$\mathcal{L}_{\text{CNN}}$: ground-truth and CNN hard-label supervision.

$$\underbrace{p_{\text{pruned},c}^{\tau} \approx p_{\text{full},c}^{\tau}}_{\text{class-token output}}, \quad \underbrace{p_{\text{pruned},d}^{\tau} \approx p_{\text{full},d}^{\tau}}_{\text{distillation-token output}}$$

$\mathcal{L}_{\text{full}}$ applies KL matching to both outputs. The original ViT acts as a teacher.

ImageNet-1K	Parameters	FLOPs	V100 speedup	Top-1 acc.
DeiT-B (paper baseline)	86M	17.6G	1.00×	83.36%
NViT-B	34M	6.8G	1.86×	83.29%

Single GPU, batch size 256; no additional 2:4 Ampere sparsity in the NViT-B row.

The structure changes while the objective encourages **similar output behavior**.

What should we retain from each pruning paper?

Paper	Core mechanism	Must remember
OBD	Diagonal curvature $\times$ weight squared	Small weight $\neq$ small damage
OBS	Inverse curvature + compensation	The remaining weights can cooperate
Han et al.	Magnitude pruning + retraining	Recovery is part of the method
Slimming	L1 on BN channel scales	Choose a variable tied to a whole structure
L_0	Expected active-gate penalty	Learn inclusion, then export zero-gated groups
NViT	Global group saliency + latency	Preserve compatible shapes; measure speed

So far: preserve **task performance** under a **computation budget**.
 Next: what happens to the **evidence** used for the prediction?

Matching targets and what they preserve

The question connecting KD and pruning

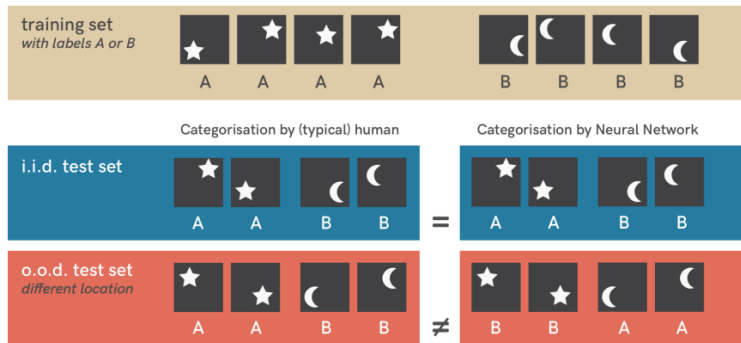
What should remain stable, and where should we apply a matching loss?

Target	Role
Output distributions	NViT's full-model KD encourages similar predictions
Hidden representations	Other KD variants can match internal features, with suitable alignment
Input evidence	EviPrune diagnoses drift and uses stable support to guide pruning

$$p_S(x) \approx p_T(x) \text{ on calibration inputs} \not\Rightarrow \mathcal{E}(f_S, x, c) \approx \mathcal{E}(f_T, x, c)$$

Matching outputs alone does not identify the input cues each model uses.

Shortcut learning: correct labels from a fragile rule



Intended rule

Recognize a star or a moon.

Learned shortcut

Use the object's location.

OOD test

Move the object to a different location.

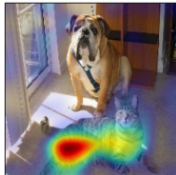
- A shortcut can work on unseen examples from the same distribution.
- A teacher may contain such a rule. KD alone does not guarantee its removal.

Useful teacher information and teacher-specific biases can both influence the student.

XAI as a diagnostic for evidence drift



Input x



Grad-CAM: cat



Grad-CAM: dog

A class-specific attribution example. These panels do not compare dense and pruned models.

For a pruning comparison

$$\mathcal{E}(f_{\text{dense}}, x, c)$$

$$\mathcal{E}(f_{\text{pruned}}, x, c)$$

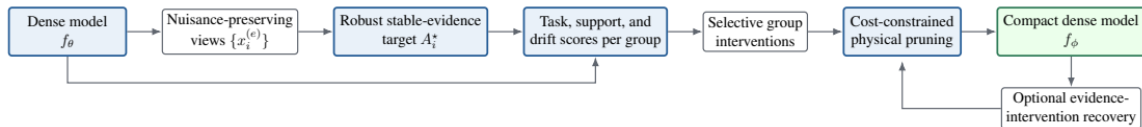
- Hold input x and target c fixed.
- Compare where attribution changes.

EviPrune's PED: average explanation divergence over samples where both models predict the target class. Inspect per-sample maps as well as the mean.

Attribution suggests where to investigate. Masking tests and held-out shifts provide additional checks.

EviPrune: use stable evidence to guide structural pruning

Ongoing research draft — method proposal, not a validated result



- ① Find evidence that recurs across reliable, semantics-preserving views.
- ② Score structural groups by task utility, stable support, and evidence drift.
- ③ Test candidate removals, then physically prune under a resource budget.

Use explanations to **propose candidates**; use interventions to **check decisions**.

Stable evidence: keep what recurs, discount what varies

$$A_i^* = \text{Norm} \left(\left[\underbrace{\text{Median}_e \tilde{A}_i^{(e)}}_{\text{recurring evidence}} - \kappa \underbrace{\text{MAD}_e \tilde{A}_i^{(e)}}_{\text{variation across views}} \right]_+ \right)$$

Align maps to the same patch grid first. MAD is median absolute deviation; $[\cdot]_+$ clips negatives to zero.

$$S_g^{\text{pre}} = \underbrace{\text{Norm}(T_g)}_{\text{task utility}} + \alpha \underbrace{\text{Norm}(U_g)}_{\text{stable-evidence support}} - \beta \underbrace{\text{Norm}(D_g)}_{\text{cross-view drift}}$$

- A **keep score**: low-scoring groups become removal candidates.
- Temporarily gate candidates off; re-rank before final budgeted removal.
- Stability depends on the chosen views and explainer; it is not causal proof.

The goal is not to copy one teacher heatmap, but to retain **reliable candidate support**.

EviPrune recovery: output KD plus evidence interventions

$$\mathcal{L}_{\text{rec}} = \underbrace{\mathcal{L}_{\text{task}}}_{\text{target labels}} + \lambda_{\text{KD}} \underbrace{\mathcal{L}_{\text{KD}}}_{\text{teacher outputs}} + \lambda_{\text{evi}} \underbrace{\mathcal{L}_{\text{evi}}}_{\text{evidence sufficiency}} + \lambda_{\text{cons}} \underbrace{\mathcal{L}_{\text{cons}}}_{\text{view consistency}}$$

- Keep the stable-support mask fixed. Compare predictions with that evidence retained versus removed.
- Encourage consistent predictions across reliable nuisance-varied views.
- This optional recovery uses prediction losses on intervened inputs rather than direct heatmap matching.

Questions to test

Does mean PED hide high-drift samples or classes?

Does the benefit survive unseen shifts at matched cost and measured latency?

Three things to take away

① **Importance is a choice.**

Magnitude, local damage, compensation, and evidence support are different criteria.

② **Sparsity is not the same as speed.**

Regularizers and gates guide selection; physical export creates smaller structure.

③ **The matching target determines what we preserve.**

KD encourages output similarity. Evidence-guided pruning asks which useful cues should survive.

Questions & Discussion

Backup: deriving the OBS update

$$\mathcal{J}(\delta w, \nu) = \frac{1}{2} \delta w^\top H \delta w + \nu (e_q^\top \delta w + w_q)$$

$$\nabla_{\delta w} \mathcal{J} = H \delta w + \nu e_q = 0 \quad \Rightarrow \quad \delta w = -\nu H^{-1} e_q$$

$$-\nu [H^{-1}]_{qq} + w_q = 0 \quad \Rightarrow \quad \nu = \frac{w_q}{[H^{-1}]_{qq}}$$

$$\delta w^* = -\frac{w_q}{[H^{-1}]_{qq}} H^{-1} e_q, \quad \Delta \mathcal{L}^* \approx \frac{w_q^2}{2[H^{-1}]_{qq}}$$

If the local model has diagonal H , the score reduces to OBD.

If $H = cI$ with $c > 0$, it ranks weights like magnitude.

Backup: the complete hard-concrete gate

$$u \sim U(0, 1), \quad s = \text{sigmoid}\left(\frac{\log u - \log(1 - u) + \log \alpha}{\beta}\right)$$

$$\bar{s} = s(\zeta - \gamma) + \gamma, \quad z = \text{clip}_{[0,1]}(\bar{s}), \quad \gamma < 0, \quad \zeta > 1$$

$$\mathcal{L}_C = \sum_j P(z_j > 0) = \sum_j \text{sigmoid}\left(\log \alpha_j - \beta \log \frac{-\gamma}{\zeta}\right)$$

$$\hat{z}_j = \text{clip}_{[0,1]}(\text{sigmoid}(\log \alpha_j)(\zeta - \gamma) + \gamma)$$

β is a temperature; $\log \alpha_j$ is learned. The last expression is the paper's deterministic test-time estimator, not the exact expected gate.

Backup: input attribution and pruning-induced evidence drift

Integrated Gradients: a baseline-relative attribution

$$\text{IG}_i(x; x') = (x_i - x'_i) \int_0^1 \frac{\partial F(x' + a(x - x'))}{\partial x_i} da$$

x : one sample; x' : baseline; i : feature index; F : one scalar model output.

EviPrune: compare explanations when predictions agree

$$\text{PED} = \frac{1}{|\mathcal{I}_{\text{agree}}|} \sum_{i \in \mathcal{I}_{\text{agree}}} \text{JS}(\mathcal{E}(f_\theta, x_i, c_i), \mathcal{E}(f_\phi, x_i, c_i))$$

Maps are nonnegative and normalized over aligned patches. Report the agreement-set size as well; PED is undefined if it is empty.

PED measures **explanation change**, not whether one explanation is more correct.

Backup: reading map and sources

Paper	Read directly	Skim first if short on time
OBD	Sec. 2.1: approximations; Sec. 2.2: recipe	Sec. 2.3 experiments
OBS	Sec. 2: constrained deletion and update	Sec. 3 inverse-Hessian computation
Han et al.	Sec. 3: regularization, pruning, retraining	Sec. 4 detailed experiments
Slimming	Sec. 3: channel scales and sparse training	Sec. 4 results; Table 1
L_0	Secs. 2.1–2.2; Fig. 2; Eqs. (12)–(13)	Sec. 2.3 and appendices
NViT	Secs. 3.1–3.3; Eqs. (6)–(9); Table 1	Sec. 4 and architecture analyses
EviPrune draft	Fig. 1; Secs. 3.2–3.3, 3.5–3.8	Fine details of Sec. 3.4

KD bridge: user-provided video, soft targets and temperature. XAI bridge: Shortcut Learning (Geirhos et al.), Fig. 2; Grad-CAM (Selvaraju et al.), Fig. 1; Integrated Gradients, Sec. 3 (backup). Full source/figure locations and Korean explanations accompany this deck.